

Installation:

To install the program (additional context can be found in the README):

1. Create and activate a virtual environment:

- If using pip, use the bash command `$python -m venv .venv`
- If using uv, use the bash command `$uv venv`

2. Activate the virtual environment:

- Windows: `$.venv\Scripts\activate`
- macOS/Linux: `$source .venv/bin/activate`

3. Install the dependencies:

- If using just pip:
 - Run the bash command: `$pip install -e .`
 - Run the bash command: `$pip install -r requirements.txt`
- If using pip and uv:
 - Run the bash command: `$uv pip install -e .`
 - Run the bash command: `$uv pip sync requirements.txt`

The Importance of Packaging:

Packaging streamlines the process of installation and gives Python a consistent way to find projects modules. Packaging also improves environment management and reproducibility; utilizing tools like uv that use the project's packaging metadata help resolve dependencies, allowing a fresh environment match a requirements file exactly.

Dependence Graph Summary:

The dependence graph on page 3 reflects the organization of my project around a few core modules connecting scraping, database, testing, and AI cleaning components. The scraping code (src.scrape) mostly just uses Selenium and its WebDriver; it's inherited by src.load_data which collects and processes the data. The database side relies on psycopg, psycopg2, and SQLAlchemy, which connect the application to models and database-related modules such as src.models and src.orm_queries. A number of test modules depend on these same components, with pytest supporting tests for buttons, pages, data formatting, end-to-end integration, and database insertion. On the application side, Flask

connects the project to the web-facing functionality, while llama_cpp, Hugging Face Hub, and the model-related modules provide the infrastructure for the LLM cleaning elements.

SQL Injection Defenses:

I modified the raw SQL queries in load_data.py that create and modify the “applicants” data table, making use of the sql.SQL class, parameter placeholders, and adding inherent limits that enforce clamping. I made similar changes to the orm_queries.py script and all of the test functions that made use of a database connection.

Least-privilege DB configuration

```
CREATE ROLE app_reader
    LOGIN
    PASSWORD 'STRONG_PASSWORD_PLACEHOLDER'
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE
    NOINHERIT;

GRANT CONNECT ON DATABASE "GradCafe" TO app_reader;
GRANT USAGE ON SCHEMA public TO app_reader;
GRANT SELECT ON TABLE public.applicants TO app_reader;
GRANT INSERT, UPDATE ON TABLE public.applicants TO app_reader;
```

Using the SQL command above, I created a non-superuser role named “app_reader” that has no create or inherit privileges; only select, insertion and update privileges are granted in a limited respect, as those are the minimum permissions that user will need to dynamically update the tables with more applicant data.

Dependency Graph:

